

HELIX

Professional Documentation Export

Source: /home/runner/work/HELIX/HELIX/README.md
Generated: July 14, 2026

Table of Contents

HELIX

The Problem

The Approach

Architecture

Feature Categories

Key Features

Real-Time Background Protection

Distributed Online Learning

Packer Detection and Decompression

VirusTotal Integration

Authenticode Signature Verification

Technology Stack

Installation

Requirements

Usage

Full UI Mode

Background Guard Mode

Enable Startup on Boot

Learning Server

Configuration

Project Structure

How It Works

Performance

License

Author

HELIX

Intelligent Malware Detection Through Hybrid Static-Behavioral Analysis

HELIX is a next-generation malware detection system that combines static PE binary analysis with real-time behavioral sandboxing to identify threats before they execute. Unlike conventional signature-based antivirus engines that depend on known malware databases, HELIX uses machine learning to detect previously unseen threats by analyzing what a binary *is* and what it *does* — not just what it looks like.

Dataset available on Kaggle: <https://www.kaggle.com/datasets/eyadarshad/helix-malware-detection-features> ---

The Problem

Traditional antivirus software relies on signature matching: a database of known malicious file hashes. This approach fundamentally fails against zero-day threats, polymorphic malware, and packed binaries that mutate their signatures on every execution. The security industry needs detection systems that understand *intent*, not just *identity*.

The Approach

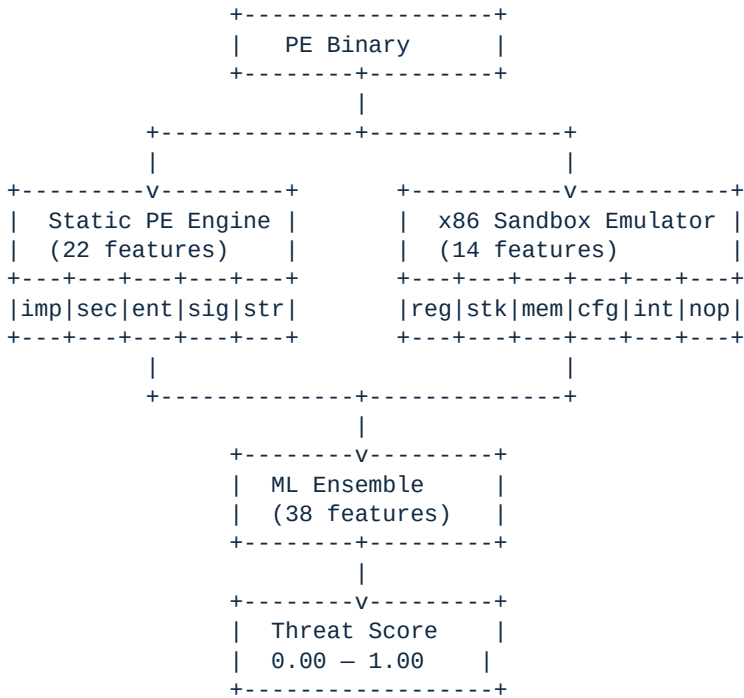
HELIX introduces a dual-layer analysis pipeline that fuses two independent intelligence sources into a single verdict:

Layer 1 — Static PE Analysis (22 features) Without executing the file, HELIX dissects the PE header, import table, section table, and binary metadata to extract structural indicators. This includes suspicious API import patterns (process injection, anti-debugging, network activity, persistence mechanisms), section entropy distribution (a strong indicator of packing or encryption), and file-level anomalies such as stripped imports, abnormal section counts, and unsigned binaries.

Layer 2 — Behavioral Sandbox (14 features) HELIX includes a custom-built x86 instruction emulator that executes binary code in a controlled environment. During execution, it traces register volatility, stack manipulation patterns, memory write density, control flow entropy, interrupt frequency, NOP sled ratios, and self-modification attempts. These behavioral signals capture runtime intent that static analysis alone cannot reveal.

Combined Classification (38 features) Both layers feed into a calibrated ensemble classifier (VotingClassifier with isotonic calibration) that produces a probability-calibrated threat score. The model outputs not just a binary safe/malware label, but a continuous confidence score that reflects the degree of threat — enabling nuanced decision-making at deployment time.

Architecture



Feature Categories

Category	Features	Signal
---	---	---
Import Analysis	Suspicious API count, injection/evasion/network/persistence imports, GetProcAddress,	
Section Analysis	Max/avg entropy, section count, writable+executable sections, zero-size sections	W
File Metadata	File size, header size, import count, export count, is_DLL, is_signed, signer trust	S
String Analysis	Suspicious string count, URL/IP pattern count	Embedded indicators of compromise
Behavioral Trace	Register volatility, stack anomaly, memory write density, control flow entropy	Wha
Evasion Detection	CPUID frequency, RDTSC checks, NOP sled ratio, self-modification	Active evasion t

Key Features

Real-Time Background Protection

HELIX runs as a background service in the Windows system tray, monitoring Downloads, Desktop, and Temp directories for new executables. When a new file appears, it is scanned automatically. Clean files receive a

silent notification. Threats trigger an immediate alert dialog with options to quarantine, delete, or allow the file.

Distributed Online Learning

Every HELIX installation connects to a central learning server by default. When a user corrects a verdict (marking a false positive as safe, or a missed threat as malicious), that correction is pushed to the server. After sufficient corrections accumulate, the server retrains the model and distributes the updated weights to all connected clients. The system becomes more accurate over time, across all deployments.

Packer Detection and Decompression

HELIX detects packed binaries by analyzing section names, magic bytes, and entropy signatures. When UPX packing is identified, the system attempts automatic decompression to analyze the real payload beneath the packer stub. For other packers, the high-entropy profile alone serves as a strong malware signal.

VirusTotal Integration

Before performing local analysis, HELIX queries the VirusTotal API with the file's SHA-256 hash. If the file is already known to the global AV community, the result is returned instantly. This provides a second opinion layer and catches known threats with zero local processing time.

Authenticode Signature Verification

HELIX validates Windows Authenticode certificate chains through PowerShell-backed verification. Signed binaries from trusted publishers receive a lower base threat score. Unsigned binaries, or those with broken certificate chains, are treated with higher suspicion.

Technology Stack

```
| Component | Technology |
|---|---|
| Language | Python 3.11+ |
| Desktop UI | PyQt6 |
| ML Framework | scikit-learn (CalibratedClassifierCV, VotingClassifier) |
| PE Analysis | pefile |
| Disassembly | capstone |
| File Monitoring | watchdog |
| Threat Intelligence | VirusTotal API |
| Learning Server | Flask |
| x86 Emulation | Custom Python-based instruction emulator |
```

Installation

BASH code block

```
git clone https://github.com/eyadarshad/HELIX.git
cd HELIX
pip install -r requirements.txt
```

Requirements

```
pefile
scikit-learn
numpy
PyQt6
requests
flask
watchdog
capstone
joblib
pandas
```

Usage

Full UI Mode

BASH code block

```
python main.py
```

Opens the HELIX desktop application with scanner, dashboard, scan history, and settings.

Background Guard Mode

BASH code block

```
python main.py --tray
```

Runs silently in the system tray, monitoring for new executables and scanning them automatically.

Enable Startup on Boot

Open Settings in the UI and enable "Start HELIX when Windows starts". This registers HELIX in the Windows startup registry to launch in tray mode automatically.

Learning Server

BASH code block

```
cd server
python helix_server.py
```

Starts the distributed learning server that accepts corrections from connected clients and periodically retrains the model.

Configuration

Edit `config.json` to customize:

JSON code block

```
{
  "virustotal_api_key": "",
  "server_url": "https://your-server.ngrok-free.dev",
  "server_api_key": ""
}
```

Key	Description
-----	-------------

---	---
-----	-----

`virustotal_api_key`	Free API key from [VirusTotal](https://www.virustotal.com/gui/join-us)
----------------------	--

`server_url`	Address of your HELIX learning server. Clear to run offline.
--------------	--

`server_api_key`	HMAC authentication key matching `HELIX_API_KEY` on the server
------------------	--

Project Structure

```
HELIX/
├── main.py                # Application entry point (UI and tray modes)
├── config.json            # Runtime configuration
├── requirements.txt       # Python dependencies
├──
├── ui/                   # Desktop interface (PyQt6)
│   ├── main_window.py    # Main application window
│   ├── scanner_thread.py # Background scan pipeline
│   ├── alert_dialog.py   # Malware detection alert
│   ├── threat_display.py # Threat gauge visualization
│   ├── tray_app.py       # System tray background guard
│   ├── label_panel.py    # User correction interface
│   ├── styles.py         # Global theme definitions
│   ├── settings.py       # Configuration management
│   └── startup.py        # Windows startup registration
├──
├── features/             # Feature extraction engine
│   ├── pe_extractor.py   # Static PE analysis (22 features)
│   └── extractor.py      # Combined feature pipeline (38 features)
├──
├── bridge/               # Analysis bridges
│   ├── disassembler.py  # PE-to-instruction disassembly
│   └── sandbox_bridge.py # Emulator orchestration
```

```

■   ■■■ trace_parser.py      # Behavioral trace analysis
■   ■■■ unpacker.py          # Packer detection and UPX decompression
■   ■■■ cert_verifier.py     # Authenticode chain validation
■   ■■■ vt_check.py          # VirusTotal API integration
■
■■■ emulator/                # Custom x86 instruction emulator
■   ■■■ cpu.py               # Register file and memory model
■   ■■■ decoder.py           # Instruction decoder
■   ■■■ executor.py          # Instruction execution engine
■
■■■ ml/                       # Machine learning pipeline
■   ■■■ online_learner.py    # Online learning and model management
■   ■■■ train.py             # Training pipeline
■   ■■■ sequence_model.py    # Sequence-based classification
■   ■■■ models/              # Trained model weights
■
■■■ server/                   # Distributed learning server
■   ■■■ helix_server.py      # Flask API with HMAC auth and rate limiting
■
■■■ sandbox_core/             # MASM-based sandbox kernel
■   ■■■ sandbox_core.asm     # x86 assembly sandbox implementation
■
■■■ asm_samples/              # Test samples (benign and malware-like)
    ■■■ benign/               # Safe assembly programs
    ■■■ malware_like/         # Simulated malicious patterns

```

How It Works

- **File Intake** — The user drops a PE file into the scanner, or the background guard detects a new download.
- **Parallel Analysis** — Two processes start simultaneously:
 - VirusTotal hash lookup (network, ~1-3 seconds)
 - Local PE feature extraction (instant)
- **Sandbox Execution** — The binary is disassembled and executed in the x86 emulator for up to 50,000 instruction steps, producing a behavioral trace.
- **Feature Fusion** — 22 static features and 14 behavioral features are combined into a 38-dimensional vector.
- **Classification** — The calibrated ensemble model produces a probability-calibrated threat score between 0.0 (safe) and 1.0 (malicious).
- **User Action** — If the score exceeds the threat threshold:
 - **Quarantine** — Moves the file to an isolated folder
 - **Delete** — Permanently removes the file
 - **Allow** — Runs the file, sends a "benign" correction to the server

- **Continuous Learning** — User corrections are pushed to the central server. After accumulating corrections, the server retrains the model and propagates improved weights to all connected clients.

Performance

Evaluated on a held-out test set of 1,755 labeled PE samples:

Metric	Score
---	---
Accuracy	99.6%
Precision	99.7%
Recall	99.6%
F1 Score	99.6%

The model correctly identifies Windows system binaries (notepad.exe, cmd.exe, explorer.exe, svchost.exe) as safe with threat scores below 0.07, while flagging malicious samples at scores above 0.99.

License

This project is provided for research and educational purposes.

Author

Eyad Arshad [GitHub](#)